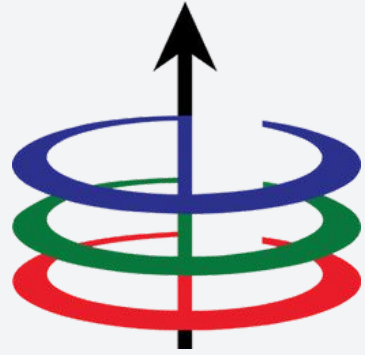




Lecture 0

Course Introduction

EEE 121 2s2526

Steven S. Sison

- Introduce yourselves
- What's with EEE 121?
- Course Policies

- **Introduce yourselves**
- What's with EEE 121?
- Course Policies

Instructor Introduction

Steven S. Sison

MEng Artificial Intelligence, BS Computer Engineering

Ubiquitous Computing Laboratory Affiliate

Smart Systems Laboratory Affiliate

Research Interests: AI/ML, Computer Vision, Agentic AI,
Robotics

Hobbies and Interests: League, Valorant, Pokemon,
Dogs, Cooking, Coffee :))



Your Turn!

Surprise Test!

Let's check if you recall your EEE 111 concepts!

What is the output of the following code:

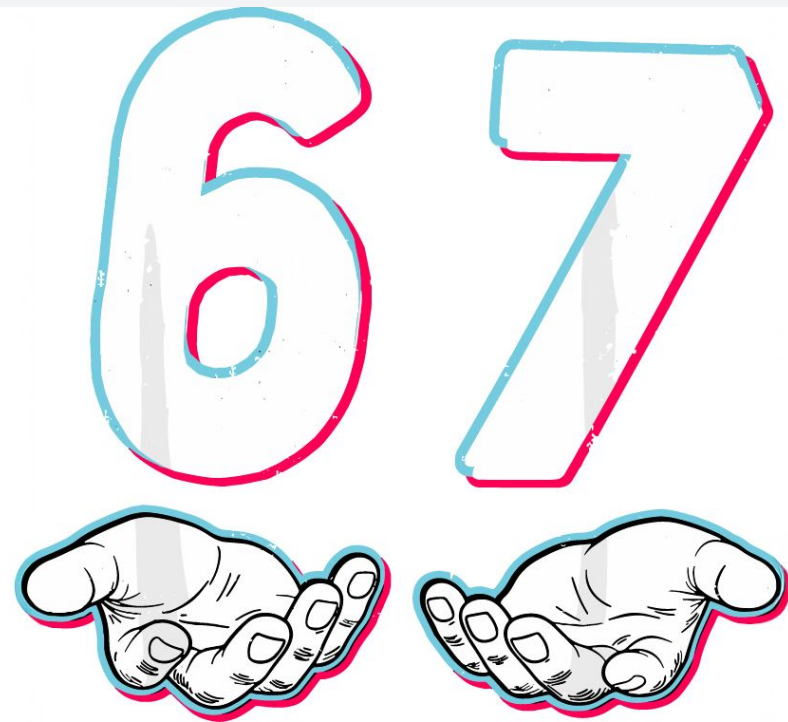
Let's check if you recall your EEE 111 concepts!

What is the output of the following code:

```
1  def function(numbers):
2      x = 0
3
4      for num in numbers:
5          x += num
6
7      return x, len(numbers)
8
9  numbers = [1, 2, 3, 4, -1, -2,-1]
10
11 result = function(numbers)
12 print(result)
```


Let's check if you recall your EEE 111 concepts!

What is the output of the following code:



```
1 def function(numbers):  
2     x = 0  
3  
4     for num in numbers:  
5         x += num  
6  
7     return x, len(numbers)  
8  
9 numbers = [1, 2, 3, 4, -1, -2, -1]  
10  
11 result = function(numbers)  
12 print(result)
```



Answer: (6,7)

- Introduce yourselves
- **What's with EEE 121?**
- Course Policies

EEE 121 deals with DSA

Data Structures

Algorithms

EEE 121 deals with DSA

Data Structures

- How data is stored and accessed
- Example:
 - Queues, Stack
 - Trees

Algorithms

EEE 121 deals with DSA

Data Structures

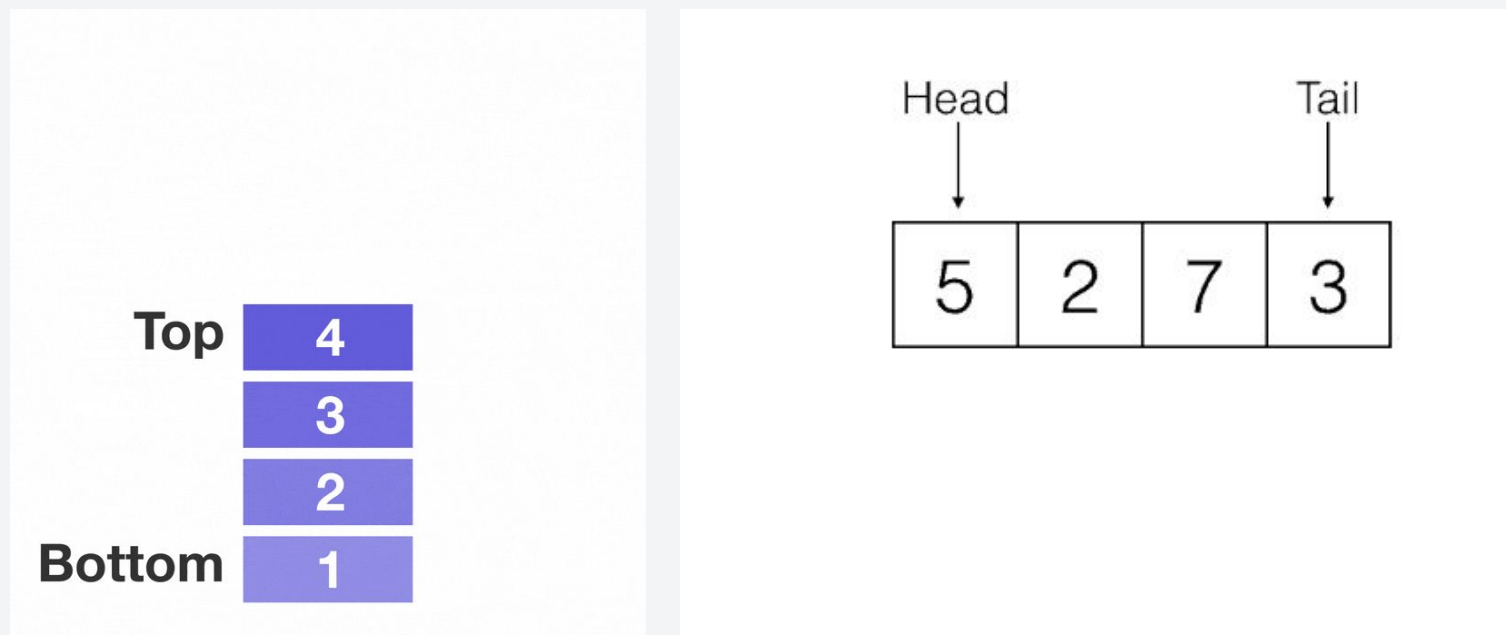
- How data is stored and accessed
- Example:
 - Queues, Stack
 - Trees

Algorithms

- Set of rules on how to solve a specific problem
- Example:
 - Dijkstra's Algorithm - finding the shortest path

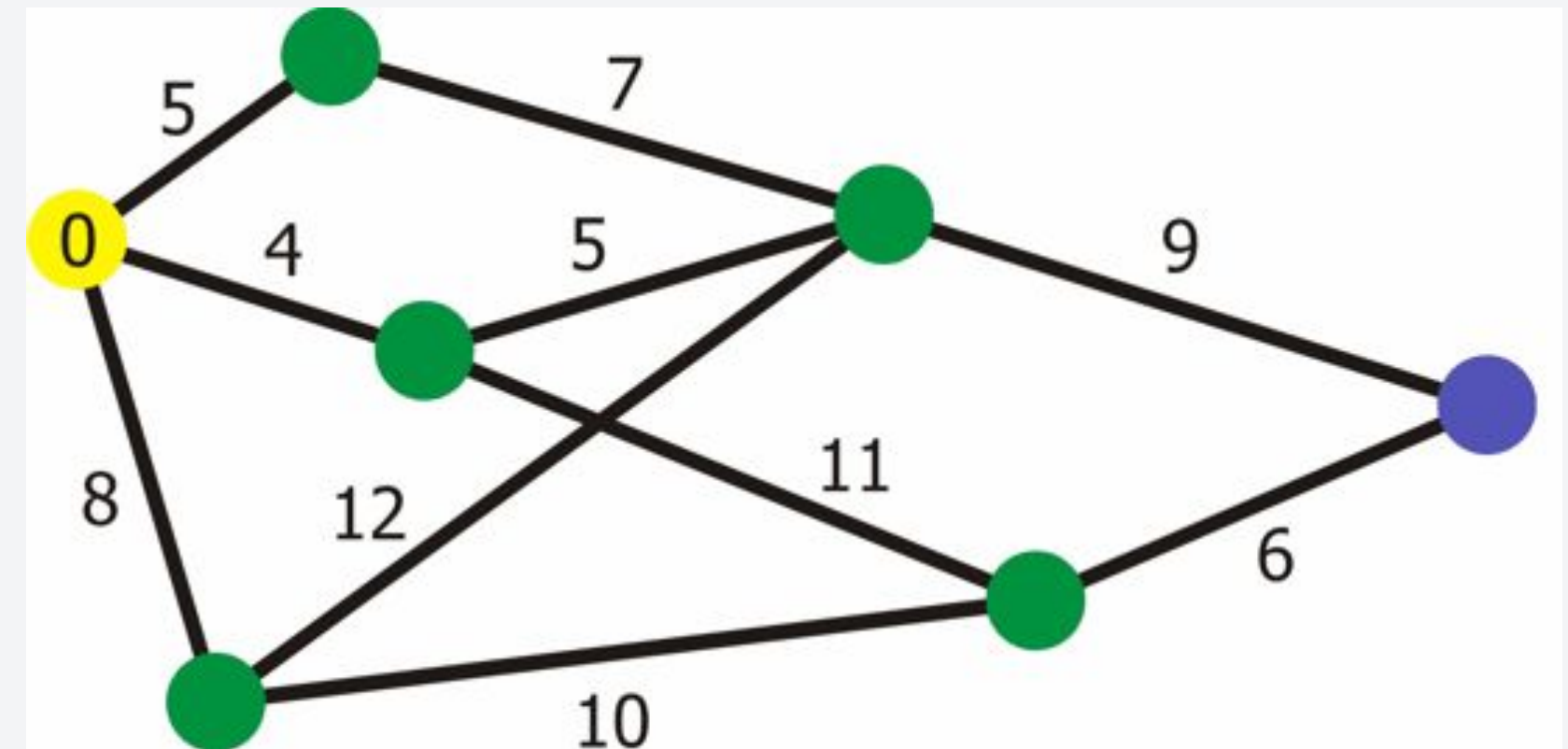
EEE 121 deals with DSA

Data Structures



Ex. Stack (Left), Queue (Right)

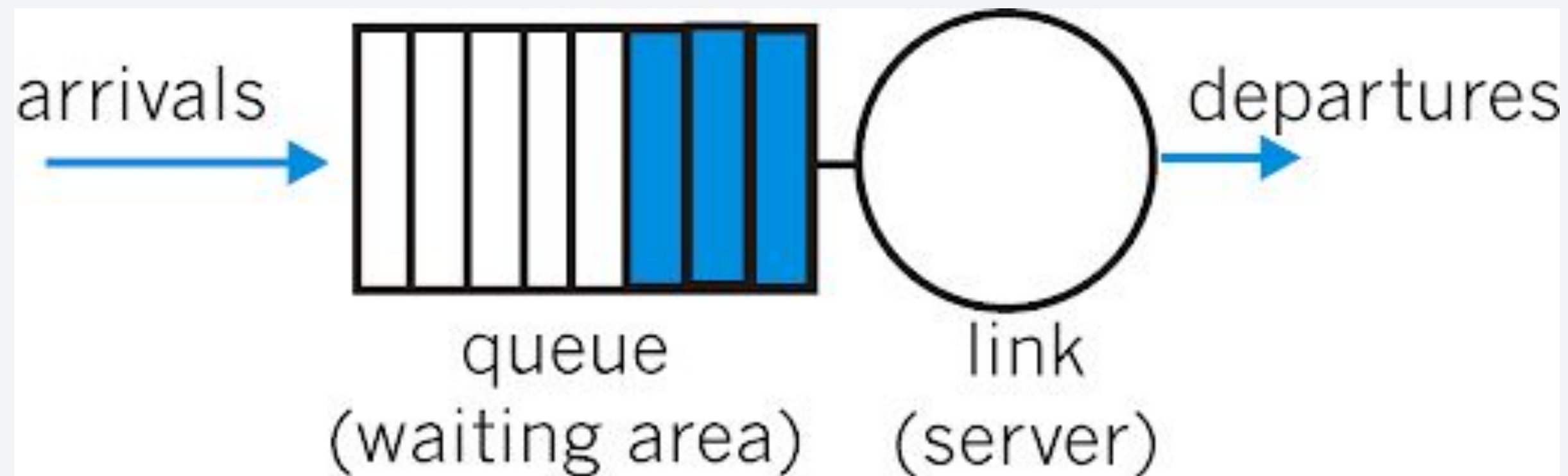
Algorithms



Ex. Dijkstra's Algorithm for path-finding.

Example: Processing Router Packets

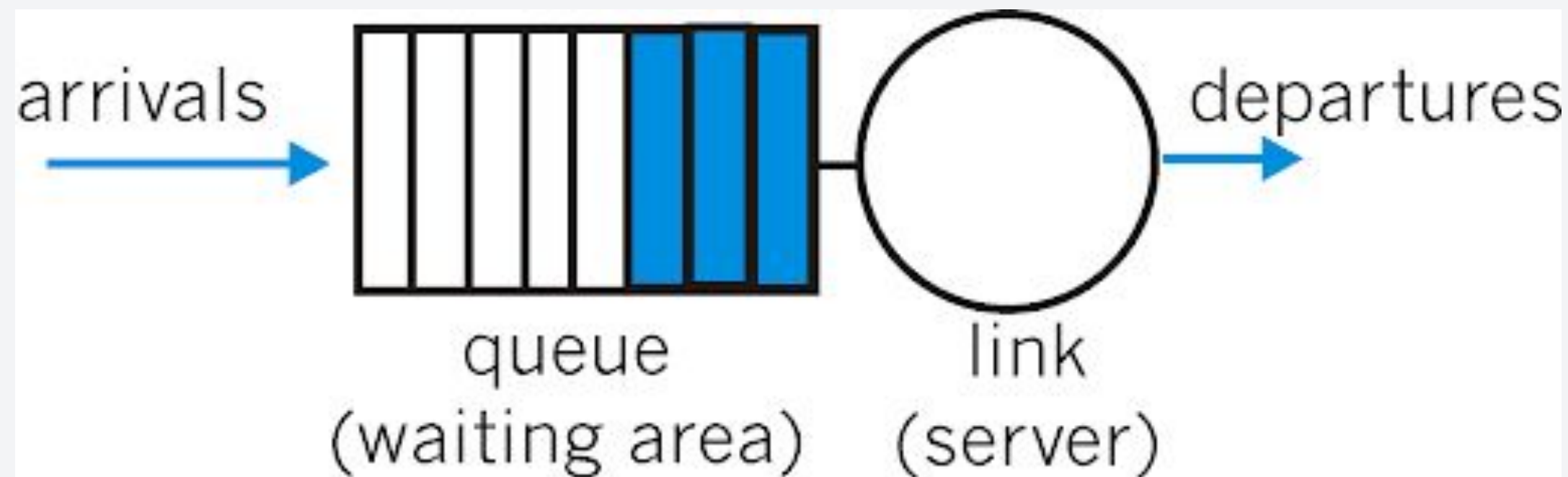
Suppose you have a router that send and receive packets (chunks of information). How do you think your router stores your data? How does it process the packets i.e. which packets are serviced first?



Example: Processing Router Packets

Method 1: Using Queues

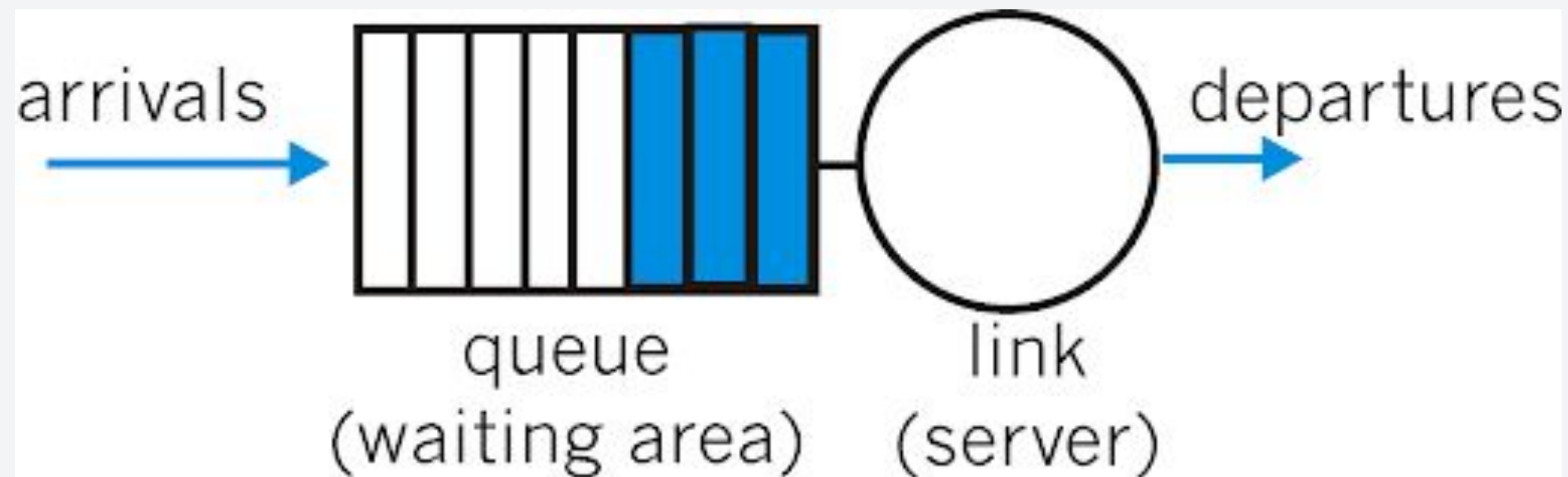
- Packets received by the router are **served in a FIFO manner (algorithm)**.
- Packets are stored in a **queue as the data structure**.



Example: Processing Router Packets

Method 1: Using Queues

- Packets received by the router are **serviced in a FIFO manner (algorithm)**.
- Packets are stored in a **queue as the data structure**.

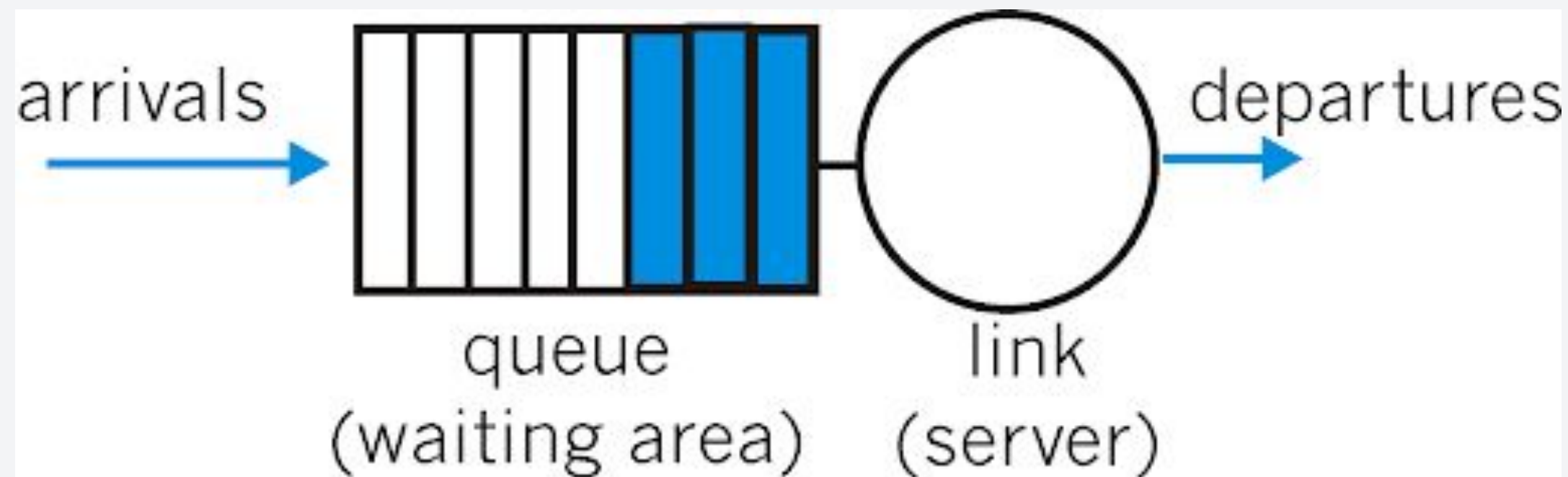


Problem: What if the queue gets full?

Example: Processing Router Packets

Method 1: Using Queues

- Packets received by the router are **served in a FIFO manner (algorithm)**.
- Packets are stored in a **queue as the data structure**.



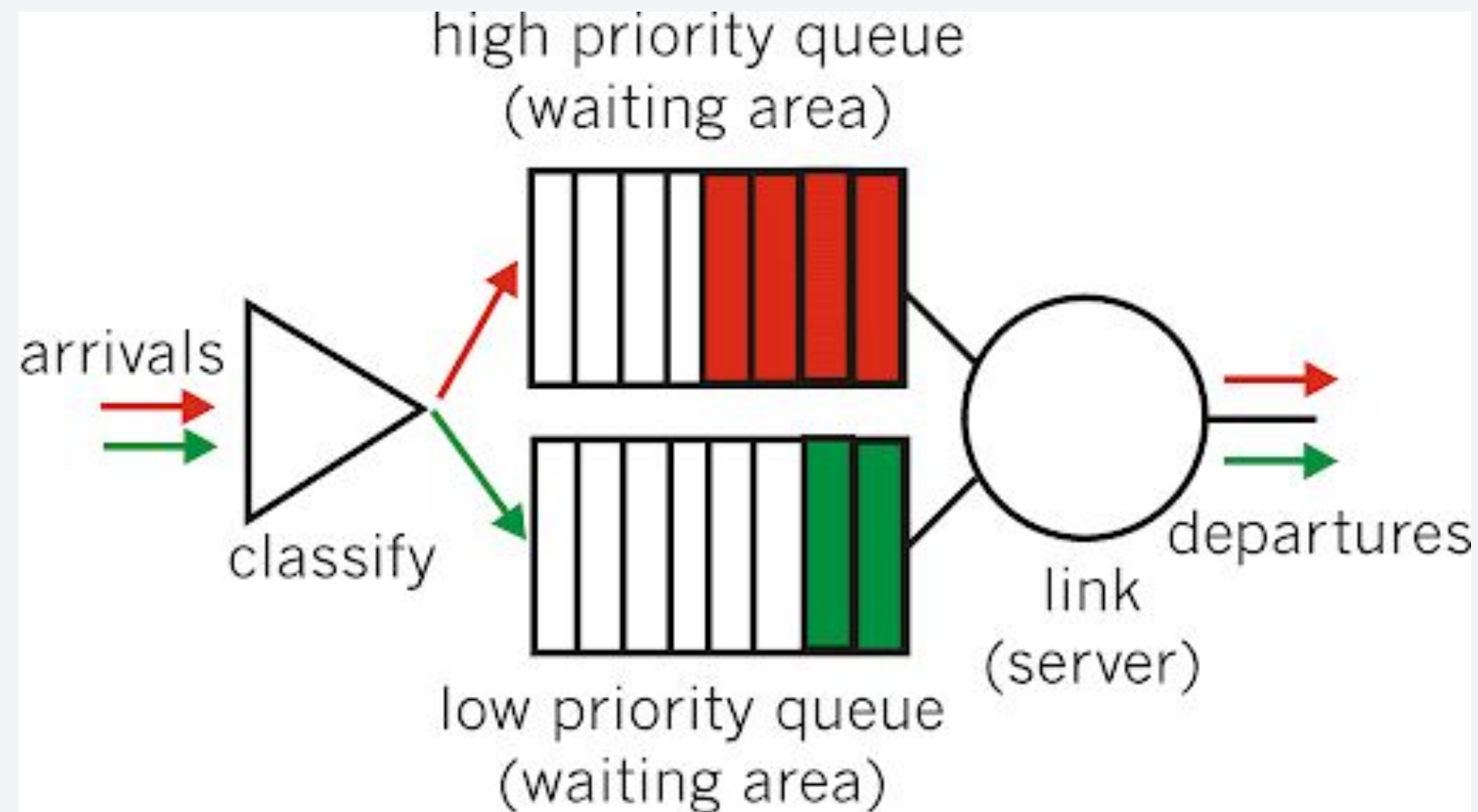
Problem: What if the queue gets full?

**Important Packets
get dropped**

Example: Processing Router Packets

Method 2: Multiple Queues with Priority-Based Scheduling

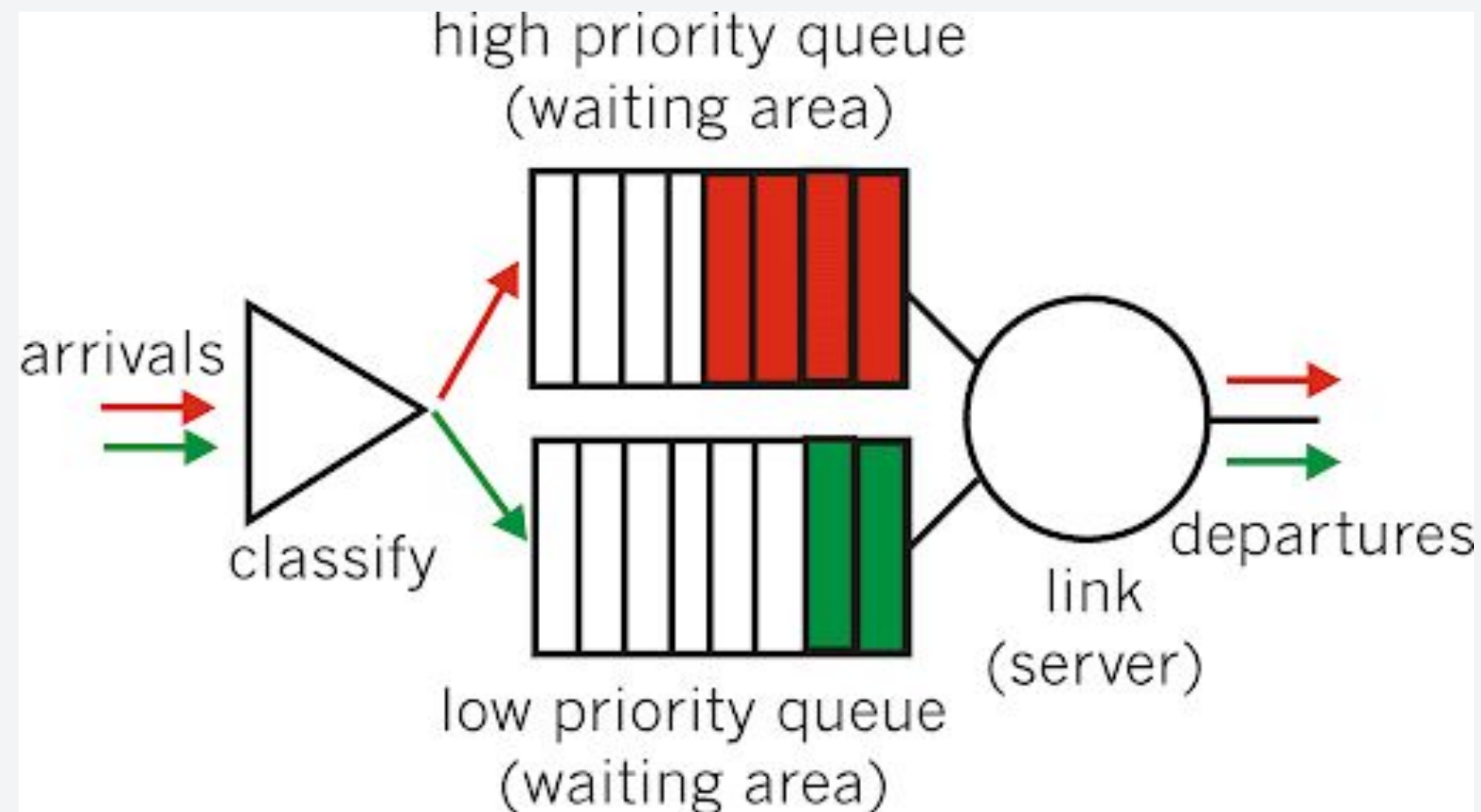
- Packets are still stored using **queues as the data structure**.
- Packets are **served using an algorithm (ex. round robin)**.



Example: Processing Router Packets

Method 2: Multiple Queues with Priority-Based Scheduling

- Packets are still stored using **queues as the data structure**.
- Packets are **served using an algorithm (ex. round robin)**.



Algorithm considerations:

1. Fairness
2. Congestion
3. Throughput

EEE 121 is not a programming class

We will focus on the synergy between data structures and algorithms.

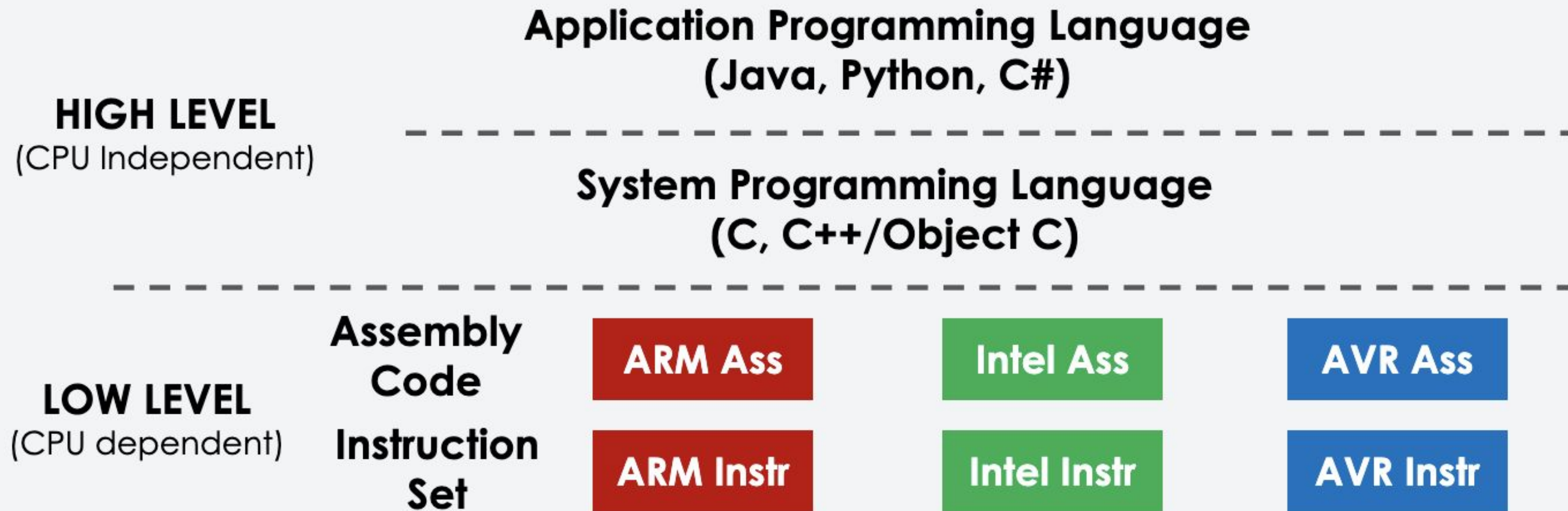
- Data structures - to **store and access data** efficiently
- Algorithms - to **solve problems** efficiently

Efficiency is the core of EEE 121

- Computational Complexity (Time and Space)

There is no perfect data structure. We just need to choose which is the most efficient for our use case.

We are going to use C++, not Python. Why?



Aspects to consider:

- Convenience vs Efficiency
- Portability and Code Reusability

We are going to use C++, not Python. Why?

Scripting with Python	System Programming with C++
Interpreted Language	Compiled Language
Has lots of use cases (web development, data science)	Reflects machine architecture (used for embedded systems, device drivers)
Suitable for rapid development	Suitable for performance intensive tasks

C++ allows developers to **handle memory allocation** making programs more efficient.

What to expect in EEE 121

- **Introduction to C++ Programming**
 - Basic C++ Libraries
 - Pointers, OOP
- **Algorithm Analysis**
 - Proving algorithm correctness
 - Time and Space Complexity
- **Recursion and Divide-and-Conquer Algorithms**
- **Linear and Non-linear Data Structures**
 - Linear Data Structures: Arrays, Linked Lists, Stacks, Queues
 - Non-linear Data Structures: Trees, Maps, Hash Table

What to expect in EEE 121

- **Graph Representation and Traversals**

- Depth-first Search, Breadth-first Search
- Shortest Path Algorithms
- Union-Find Data Structure
- Minimum Spanning Trees

- **Other Algorithms (Dynamic Programming and Greedy Algorithms)**

- Introduce yourselves
- What's with EEE 121?
- **Course Policies**

Communication Plan and Logistics

Email: steven.sison+eee121@eee.upd.edu.ph.

Use the subject [EEE 121 *Section*] - *Concern*.

Discord: Announcements, questions, class concerns

UVLe: Course materials, submission bins

Class Schedule: WF 10:00-11:00AM Lec 11:00-12:30PM Lab (WFUV1)

WF 1:00-2:00PM Lec 2:00-3:30PM Lab (WFWX1)

Modality: Assume F2F unless otherwise stated

Gameplan:

- Wednesday - lecture
- Friday - lab (weekly activities, software project)

Grading System

Weekly Activities	40%	← Lab
Software Project	30%	
Long Exam	30%	← Lec

NOTE. You are required to achieve **at least 50%** on each component (Lec & Lab)

Weekly Activities. Essay, coding activities (with documentation).

Software Project. Coding project that may or may not require a documentation.

Long Exam. Two exams (midterm and final). Tentative Dates: March 23 (Midterms), May 25 (Finals)

About Late Submissions

For late submissions, we will be using the following formula:

$$\text{score} = Ae^{-(t/30.37)}$$

Where t = number of days that elapsed after the deadline, A is your score.

Note. I will only be accepting submissions up until **4 weeks** after the deadline.

Course Materials and Calendar of Activities

Syllabus. Access through this [link](#). Alternatively, the syllabus is also sent through email and discord.

Calendar. Our weekly calendar is also shown in the syllabus. For a more detailed view, you can also go through this [link](#).

Class Rules

Attendance. Attendance will be recorded either before or after class (surprise para di niyo alam :)).

Note: Only 6 absences are allowed, 3 late = 1 absence.

Consultation. You may book through this [link](#).

Intellectual Dishonesty. Honor before excellence.

Intellectual Dishonesty

While we encourage you to collaborate with your peers, we do not tolerate any forms of plagiarism and academic dishonesty.

- Code submissions for your weekly activities and software project will be passed through a similarity checker.
- It is acceptable to use code snippets from the internet, provided that you cite your source appropriately. However, you are not allowed to completely copy or plagiarize your whole work from the internet.

Students who are flagged will receive disciplinary action.

On the Acceptable Use of AI

I allow the use of AI for the understanding and enrichment of our course materials. However, there are limitations.

- You are allowed to use AI to understand concepts that are unclear to you. However, I highly recommend to consult with the instructor or the SA first.
- You can consult AI models for C++ syntax. However, I will not allow any submissions that are AI generated specially in your code and documentation.

Any questions?

Activity: Hello world!

Activity 1: Hello world!

For a more detailed installation guide, you can visit this [link](#).

1. Install your c++ compiler using this [link](#) for windows. For mac users, a c++ compiler is already installed in your machine.
2. Install your IDE, preferably VSCode. You can install the C++ extension if you want, but it will be take home.
3. Create a new file with filename <student_no>.cpp and put the code shown in the next slide.
4. Open a terminal, compile your code, and run using these commands:
 - a. `g++ filename.cpp -o "filename"`
 - b. `./filename`
5. Send a screenshot of your output in our discord server! :D

Activity 1: Hello world!

Make sure to include these into your system environment variables:

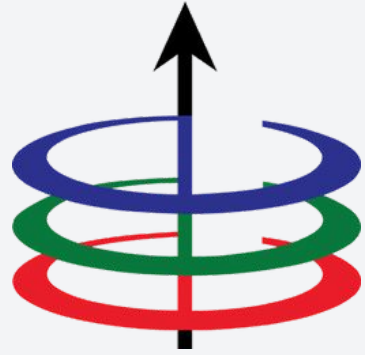
- C:\msys64\ucrt64\bin
- C:\msys64\mingw64\bin
- C:\msys64\usr\bin

Activity 1: Hello world!

```
#include <iostream>

int main()
{
    std::cout << "Hello World" << std::endl;
}
```

Copy



Lecture 0

Course Introduction

EEE 121 2s2526

Steven S. Sison